Name: Nelson Puthuraja Nadar **Roll no:** 21 **Class:** SE/INFT-B **Batch:** B-1

Q7) Laboratory Exercise

A) Procedure:

a) Answer the following

- **Explain Version Control and its types**
  - Version control is a system that records changes made to files over time so that you can check older versions, see who made changes, and avoid losing important work. It is widely used by software developers to manage code and work together safely.
  - Version control is useful because it helps you keep a history of changes, compare old and new versions, undo mistakes, and work in teams without conflicts.
  - There are three main types of version control. Local version control stores all changes only on your own computer. Centralized version control uses a single main server where all code is stored, and users send and receive changes from that server. Distributed version control gives every user a full copy of the entire project and its history, so they can work offline and share changes later.
  - Some popular tools include Git, which is the most widely used today. Platforms like GitHub, GitLab, and Bitbucket allow teams to host and collaborate on Git repositories.

- **What is Git and GitHub?**
  - Git is a distributed version control system that lets developers track changes in their code over time. With Git, every user has a full copy of the project which makes it easy to work offline, check older versions, undo mistakes, and merge work from different people.
  - GitHub is an online platform that stores Git repositories on the internet so that teams can share their code and work together. GitHub also provides extra features such as issue tracking, pull requests, project discussions, and collaboration tools. In simple words, Git handles version control on your computer, and GitHub helps you use Git on the internet for teamwork.

- **Explain different Git Commands**

1. Initialize a Git Repository

- The command git init is used to create a new Git repository inside the current directory. This sets up the hidden .git folder that Git uses to track changes.

Command: git init

Meaning: Creates a new Git repository in the current directory.

Example: git init

Output: Initialized empty Git repository in /path/to/repo/.git/

2. Check Status of Files

- The command git status shows the current state of files in the repository. It tells you which files are untracked, staged, or modified. This is useful for understanding what has changed before making a commit.

Command: git status

Meaning: Shows which files are tracked, untracked, staged, or modified.

Example: git status

Output:

```
Untracked files:
file1.txt
Changes not staged for commit:
modified: file2.txt
```

3. Add Files to Staging Area

- The command git add <filename> is used to send files to the staging area. This means the files are prepared and ready for the next commit. Adding does not produce visible output.

Command: git add <filename>

Meaning: Stages files to prepare them for the next commit.

Example:

```
git add file1.txt
```

Output: (No output)
To check staged files, you can use git status. The output shows:
Changes to be committed:
new file: file1.txt

4. Commit Changes

- The command git commit -m "Commit message" saves the staged changes to the local repository. The message describes what was changed.

Command: git commit -m "Commit message"

Meaning: Saves staged changes to the local repository.

Example:
git commit -m "Added file1.txt"
Output:
[main 1a2b3c4] Added file1.txt
1 file changed, 1 insertion(+)
create mode 100644 file1.txt

5. View Commit History

- The command git log lets you see the list of past commits, along with commit IDs, authors and timestamps.

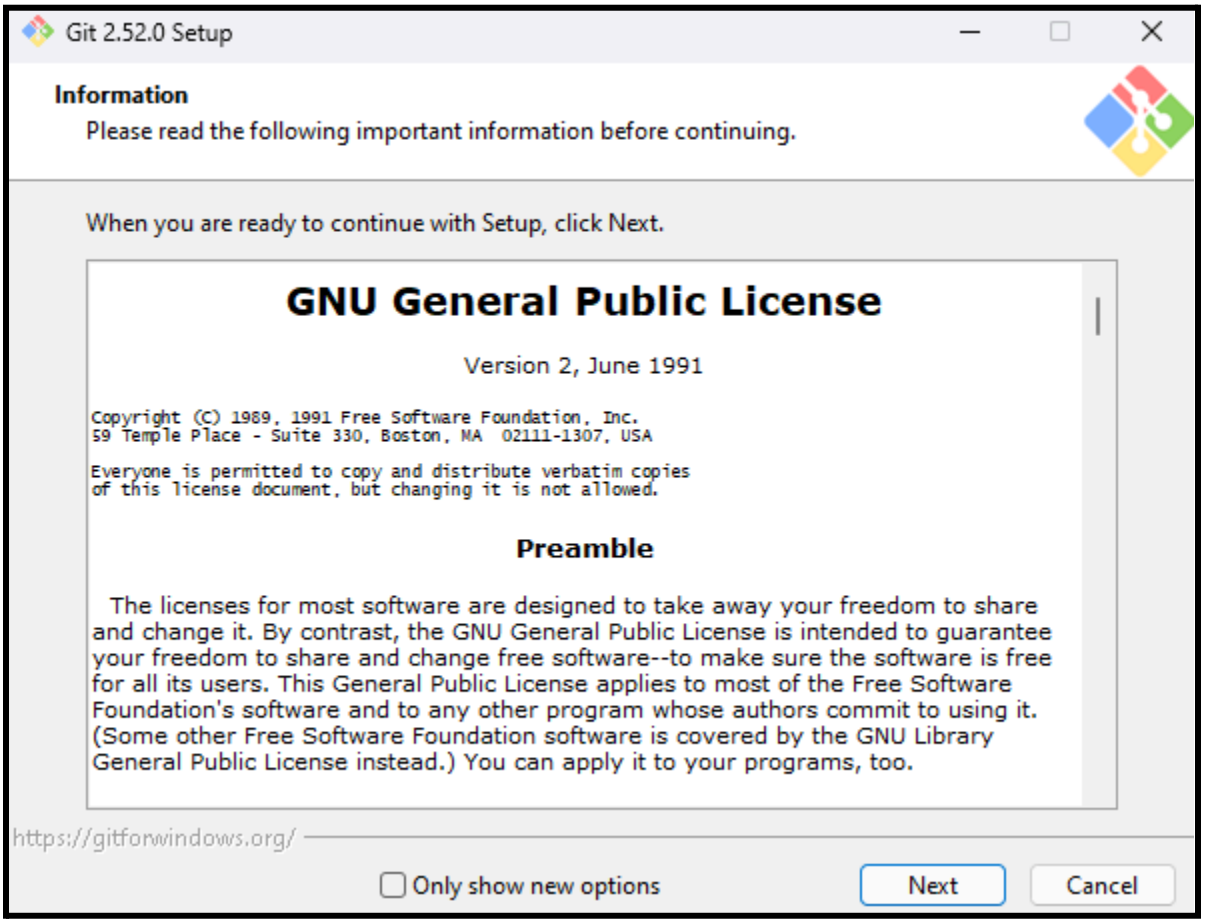
Command: git log

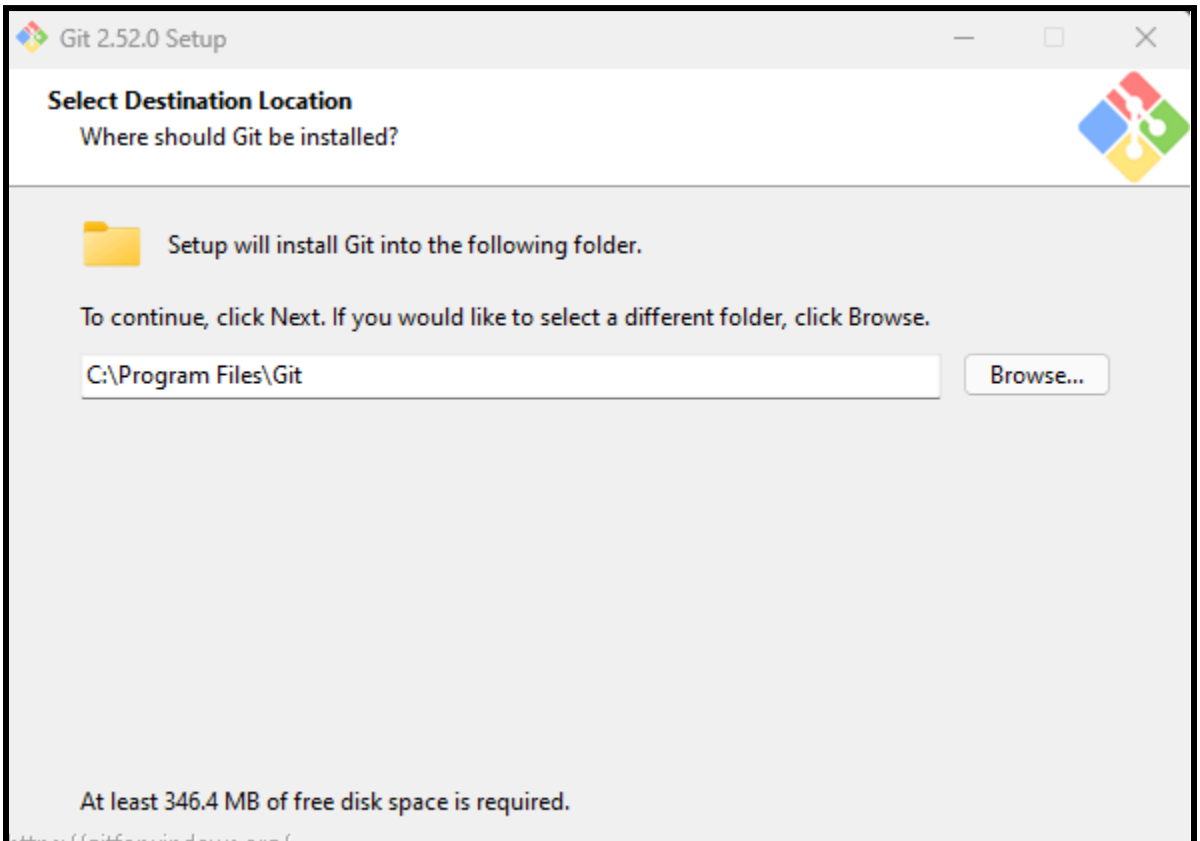
Meaning: Shows history of commits.

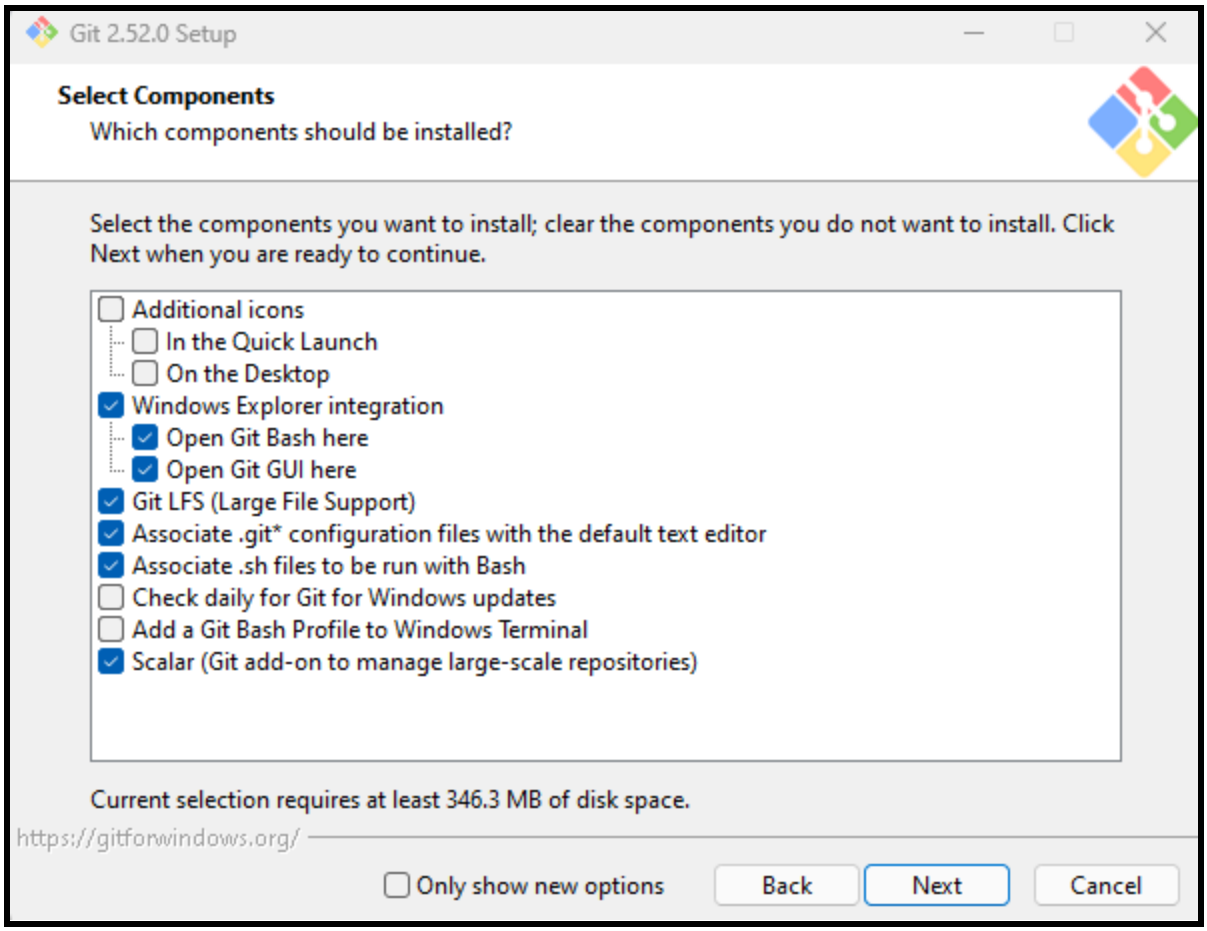
Example:
git log
Output:
commit 1a2b3c4 (HEAD -> main)
Author: User
Date: ...
Added file1.txt

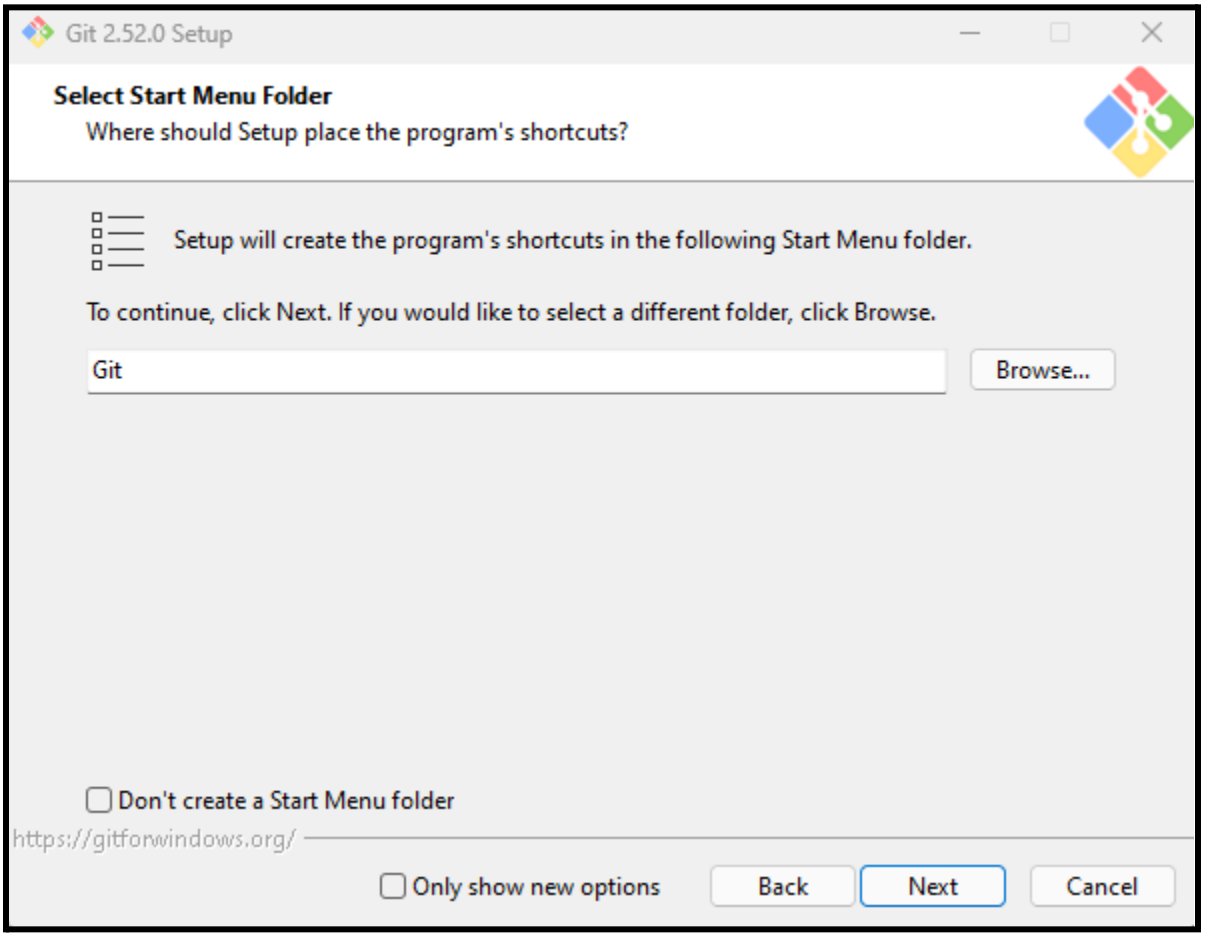
b) Execute following on Git and GitHub and attach screenshots:

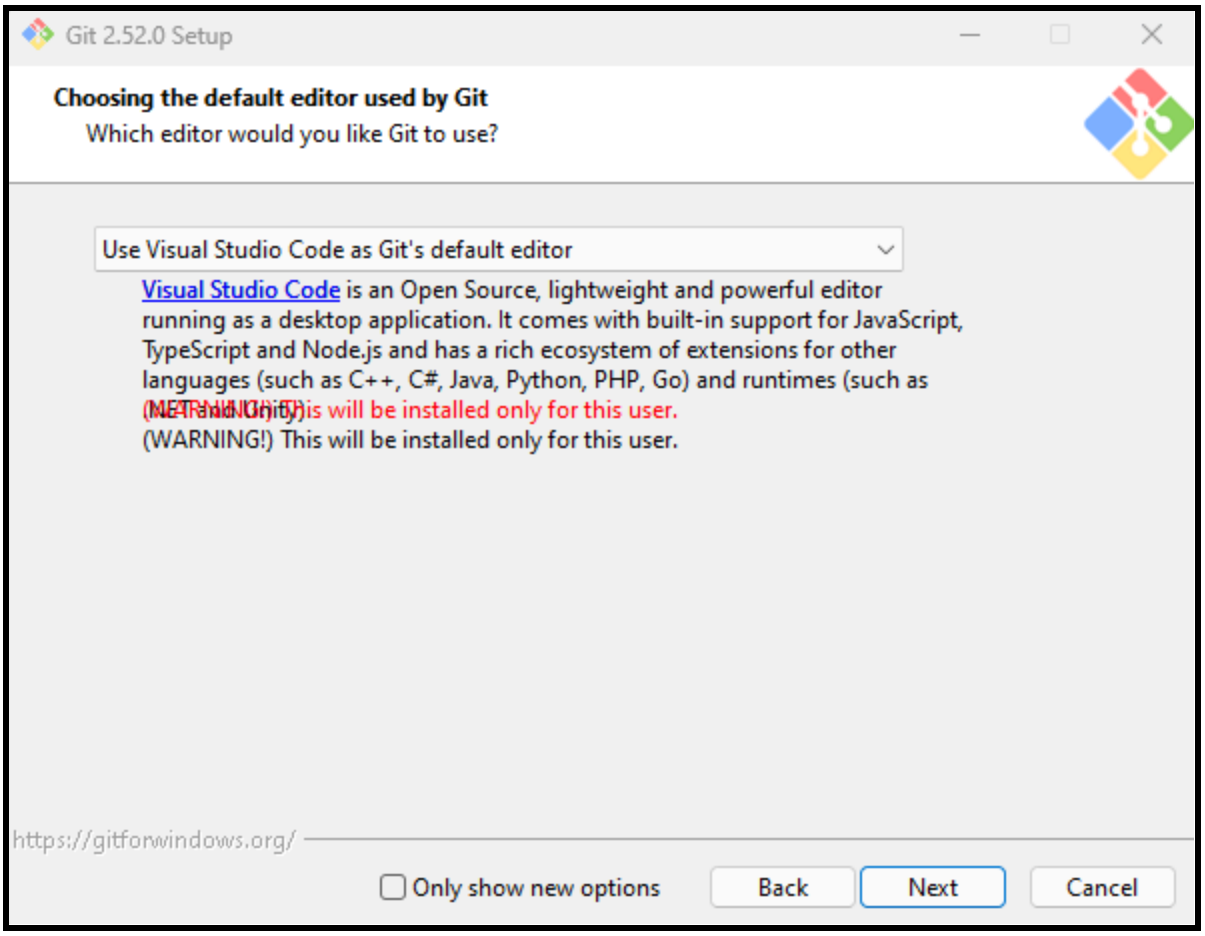
Git Installation Process

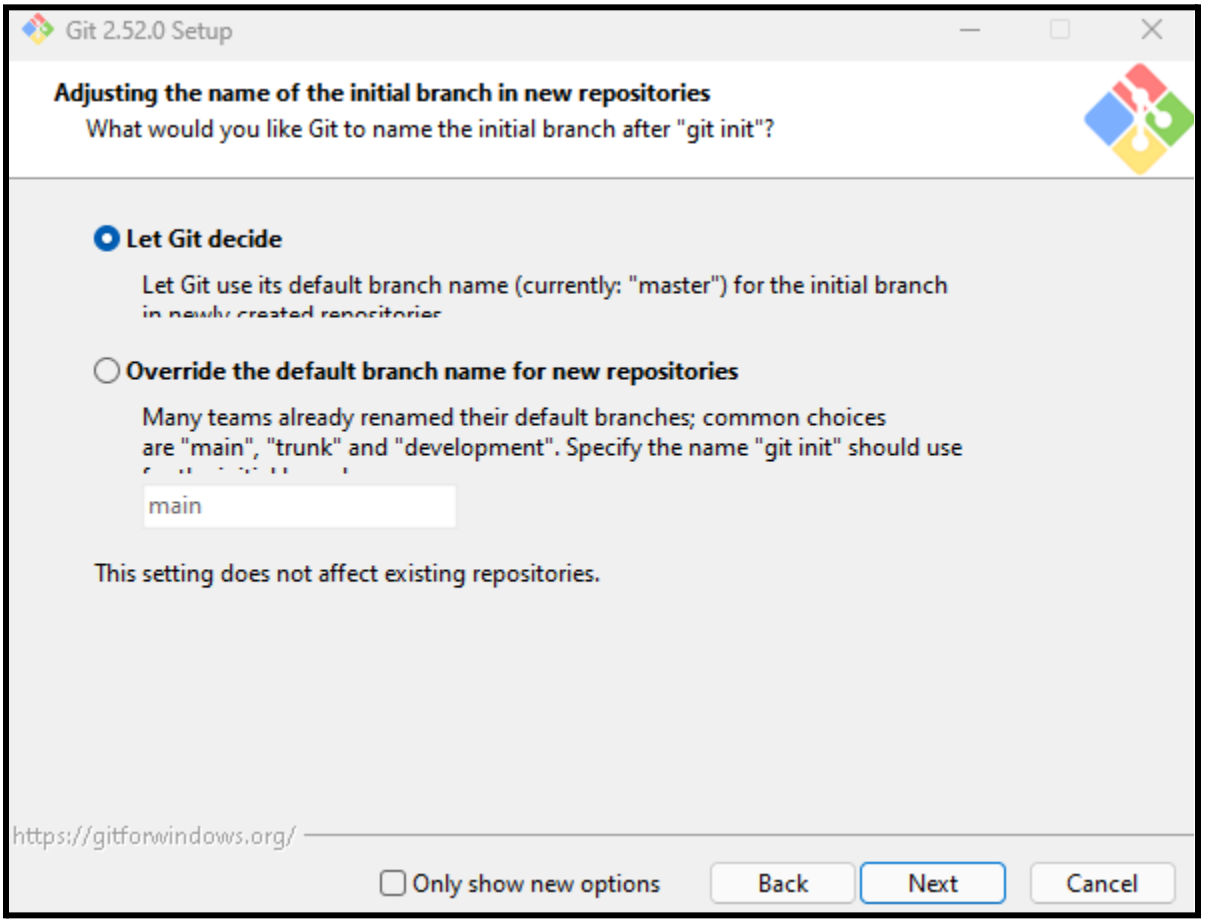


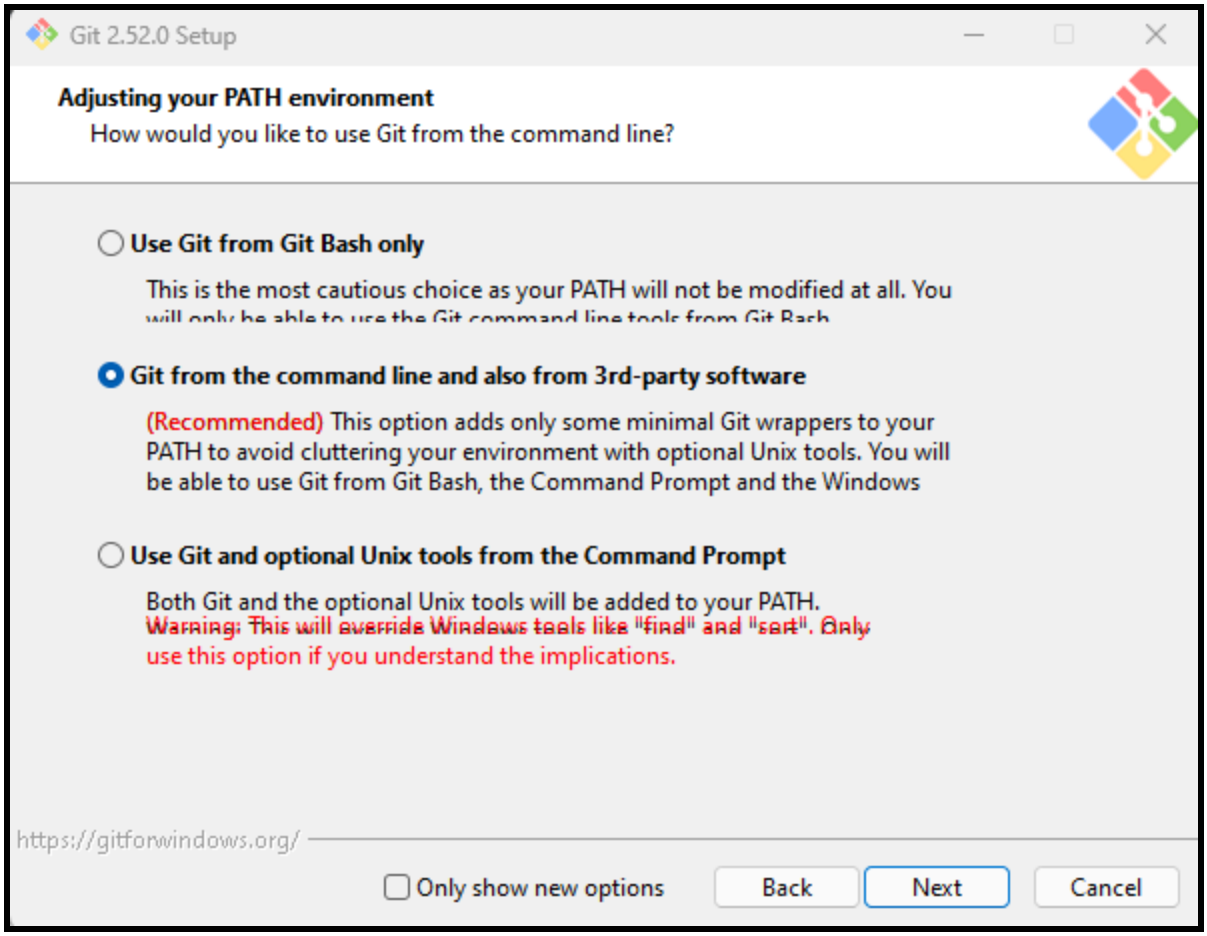


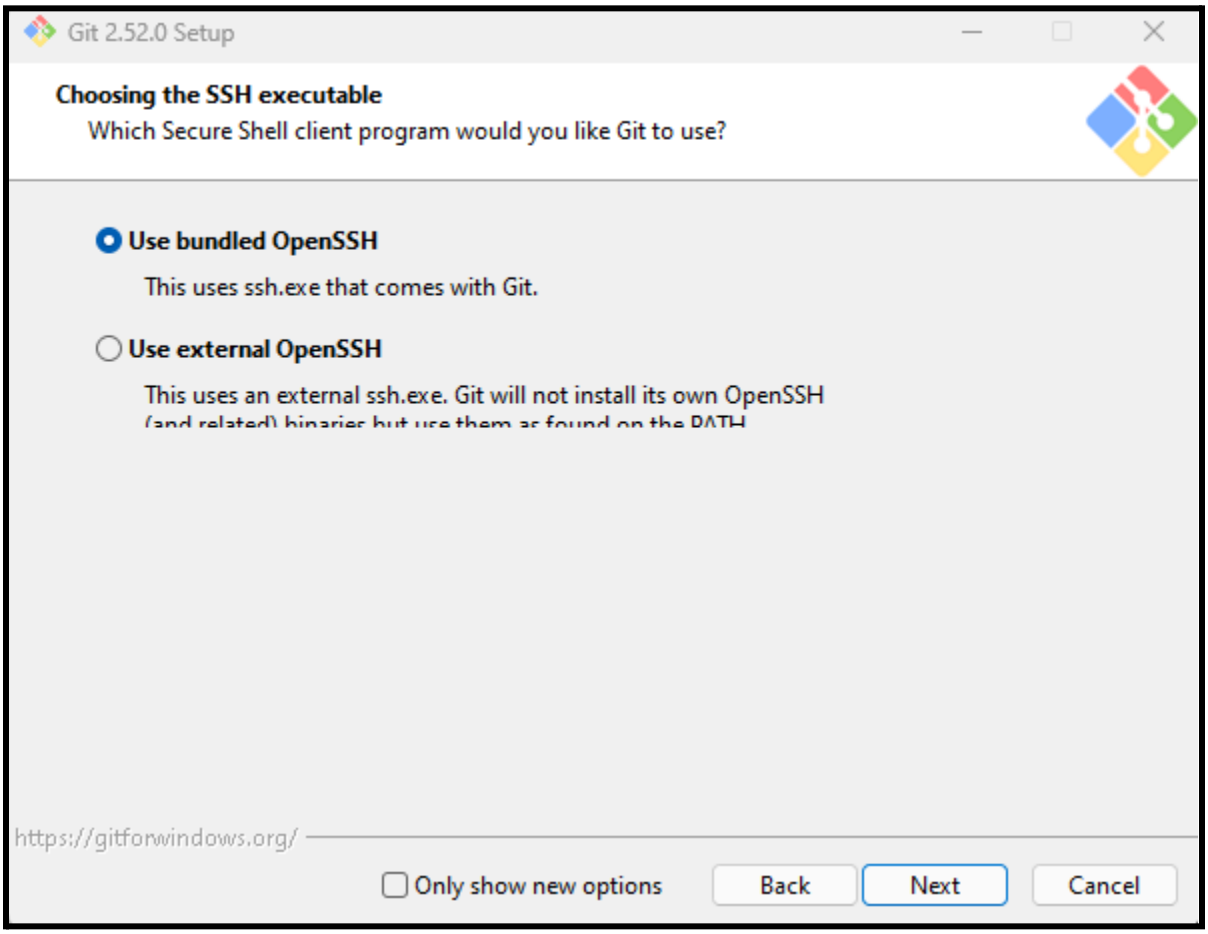

Git 2.52.0 Setup
Select Components
Which components should be installed?
Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.
Additional icons
In the Quick Launch
On the Desktop
Windows Explorer integration
Open Git Bash here
Open Git GUI here
Git LFS (Large File Support)
Associate .git* configuration files with the default text editor
Associate .sh files to be run with Bash
Check daily for Git for Windows updates
Add a Git Bash Profile to Windows Terminal
Scalar (Git add-on to manage large-scale repositories)
Current selection requires at least 346.3 MB of disk space.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel

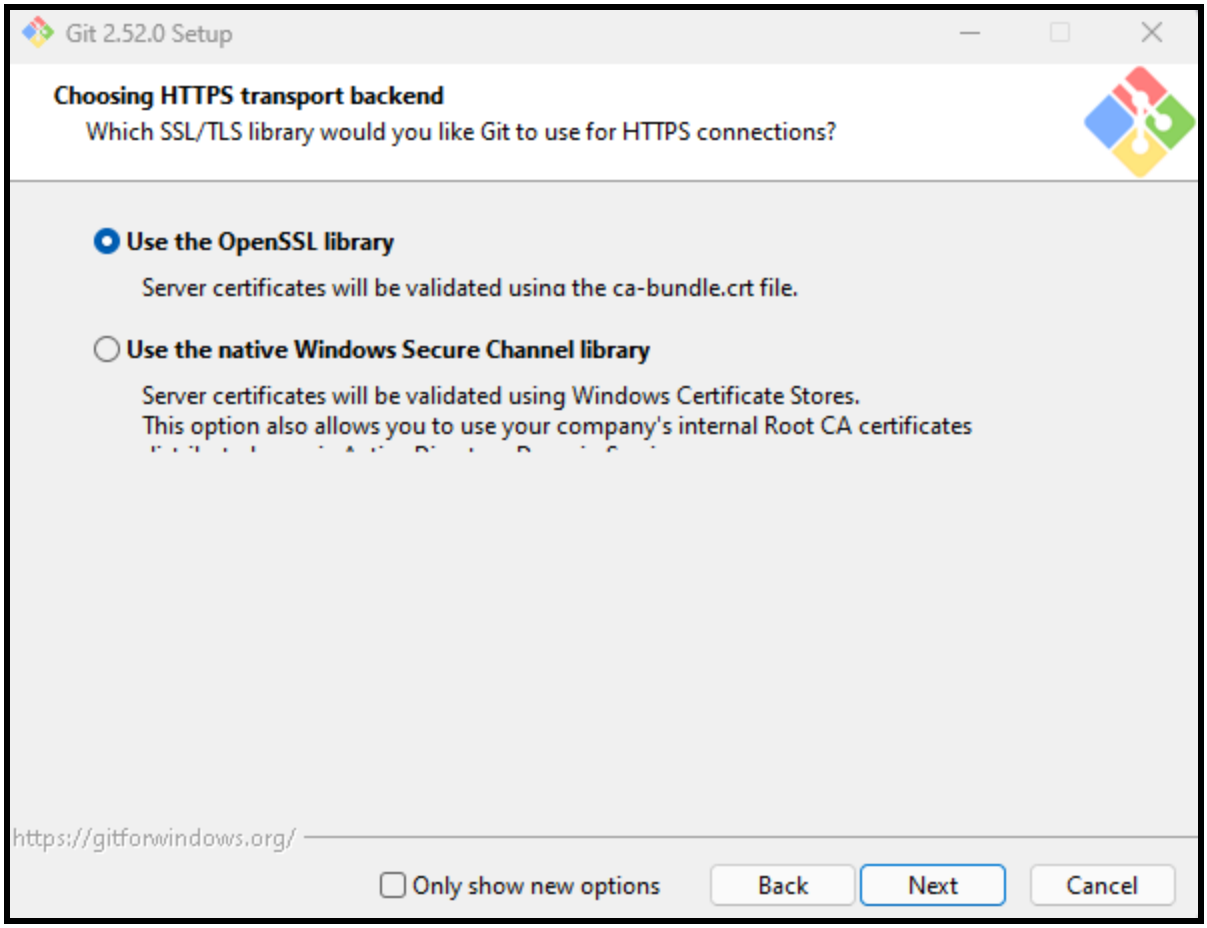

Git 2.52.0 Setup
Select Start Menu Folder
Where should Setup place the program's shortcuts?
Setup will create the program's shortcuts in the following Start Menu folder.
To continue, click Next. If you would like to select a different folder, click Browse.
Git
Browse...
Don't create a Start Menu folder
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Choosing the default editor used by Git
Which editor would you like Git to use?
Use Visual Studio Code as Git's default editor
Visual Studio Code is an Open Source, lightweight and powerful editor running as a desktop application. It comes with built-in support for JavaScript, TypeScript and Node.js and has a rich ecosystem of extensions for other languages (such as C++, C#, Java, Python, PHP, Go) and runtimes (such as .NET and Unity).
(WARNING!) This will be installed only for this user.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch
Override the default branch name for new repositories
Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use
main
This setting does not affect existing repositories.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use external OpenSSH
This uses an external ssh.exe. Git will not install its own OpenSSH
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates
https://gitforwindows.org/
Only show new options
Back
Next
Cancel

Git 2.52.0 Setup

Configuring the line ending conversions

How should Git treat line endings in text files?

- (●) **Checkout Windows-style, commit Unix-style line endings**

  Git will convert LF to CRLF when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects,

- ( ) **Checkout as-is, commit Unix-style line endings**

  Git will not perform any conversion when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects,

- ( ) **Checkout as-is, commit as-is**

  Git will not perform any conversions when checking out or committing text files. Choosing this option is not recommended for cross-platform

https://gitforwindows.org/

[ ] Only show new options | Back | Next | Cancel

Git 2.52.0 Setup

Configuring the terminal emulator to use with Git Bash

Which terminal emulator do you want to use with your Git Bash?

- (●) **Use MinTTY (the default terminal of MSYS2)**

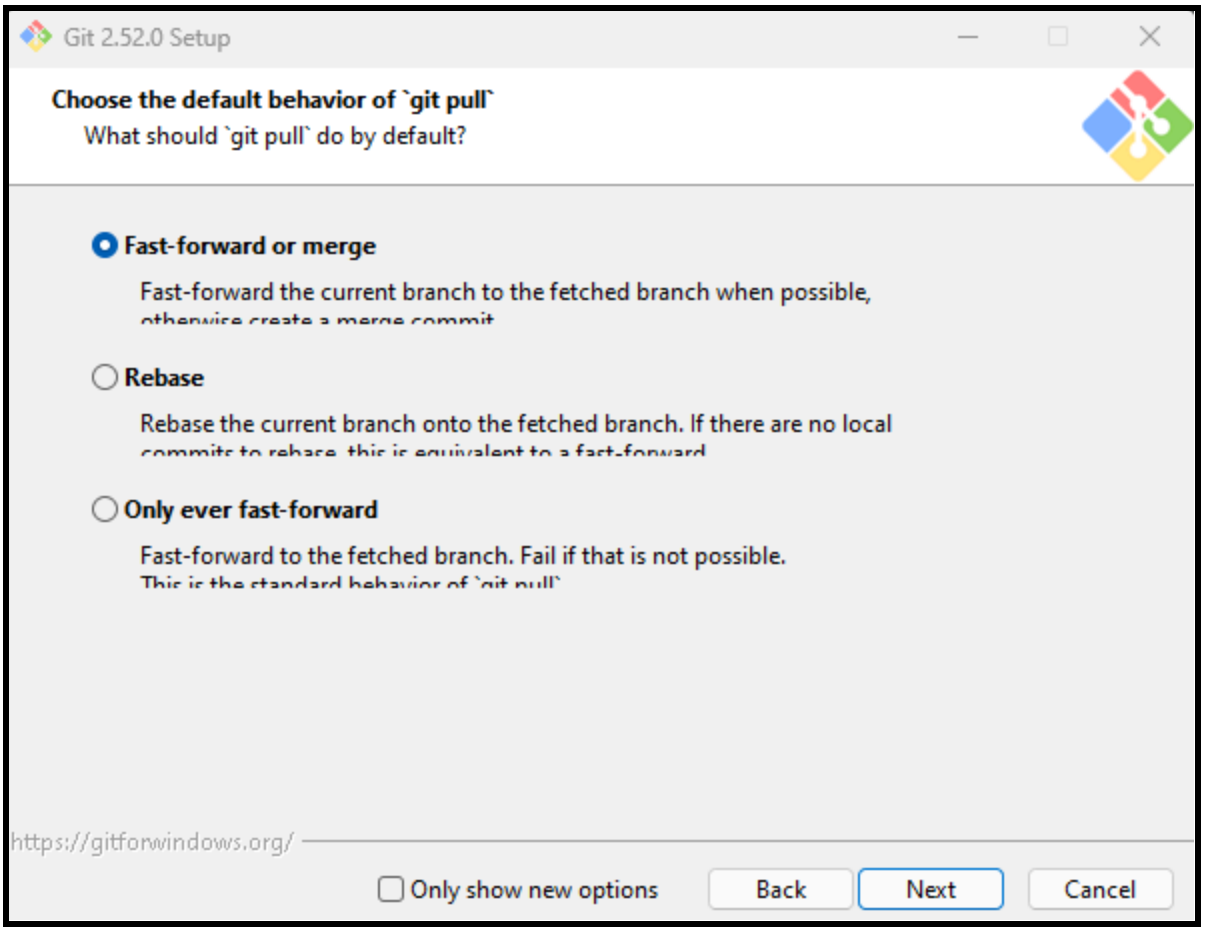
  Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work

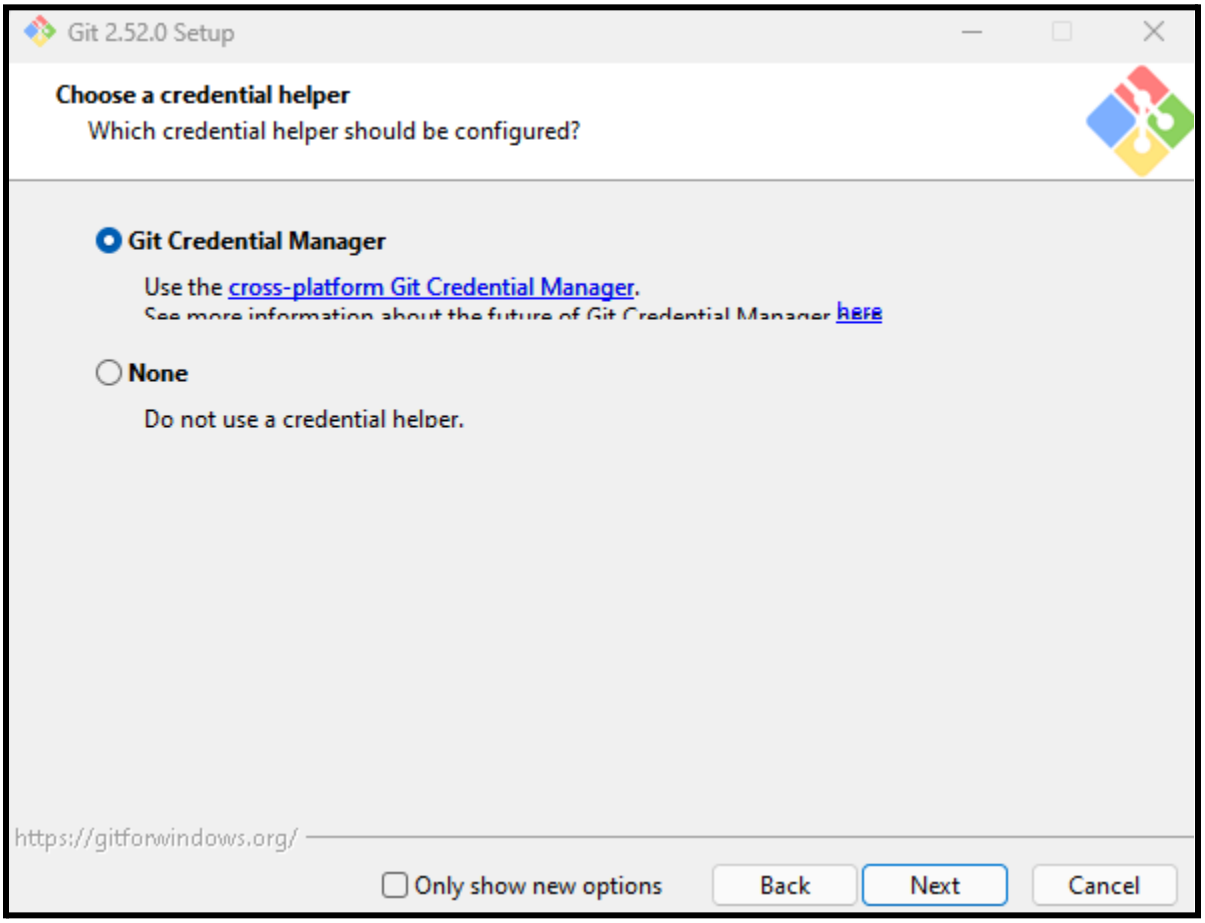
- ( ) **Use Windows' default console window**

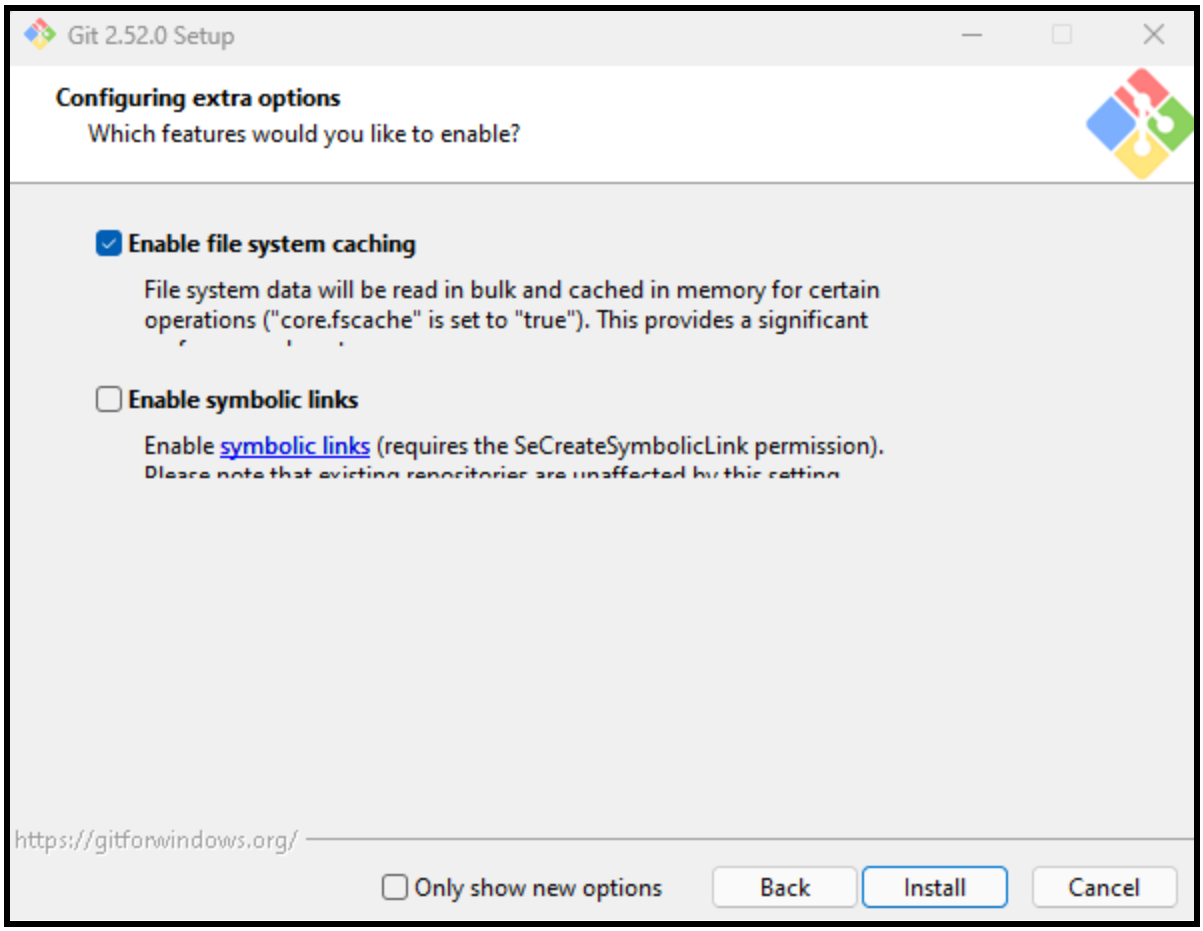
  Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed

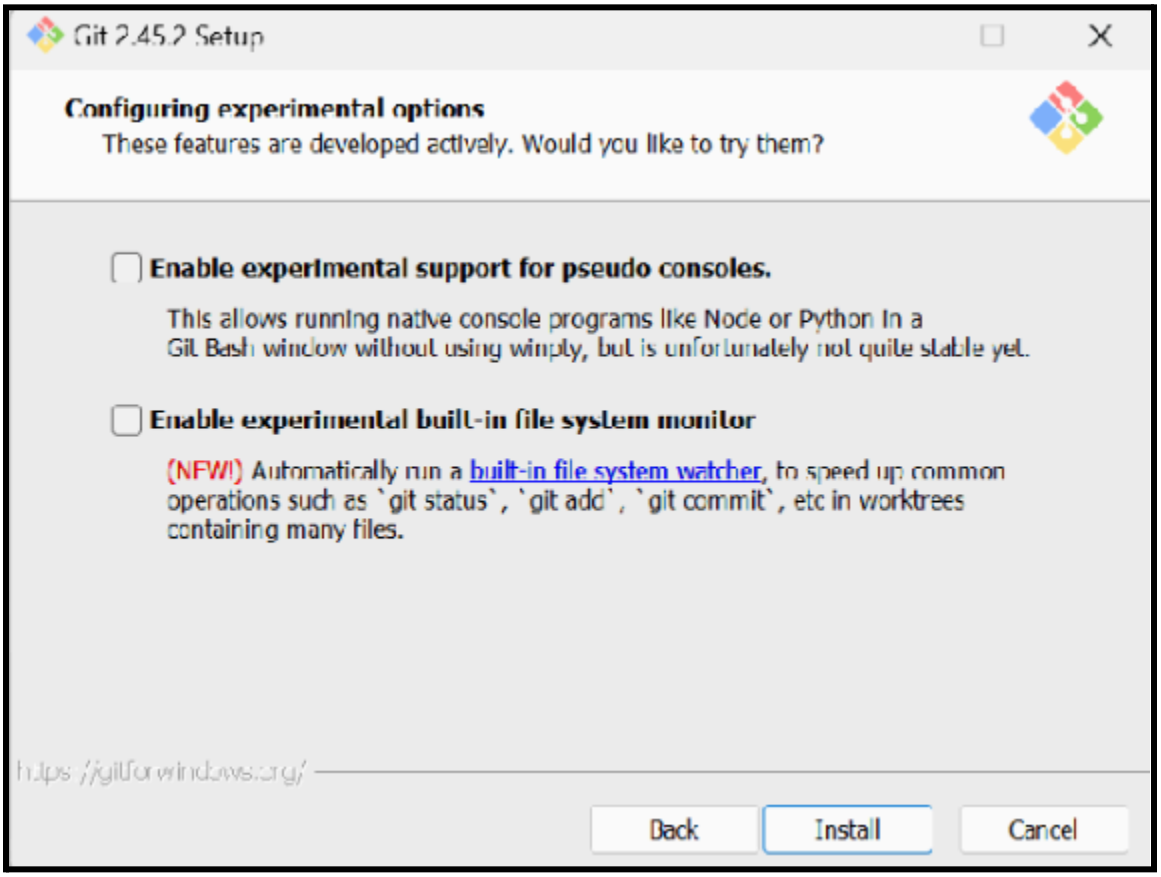
https://gitforwindows.org/

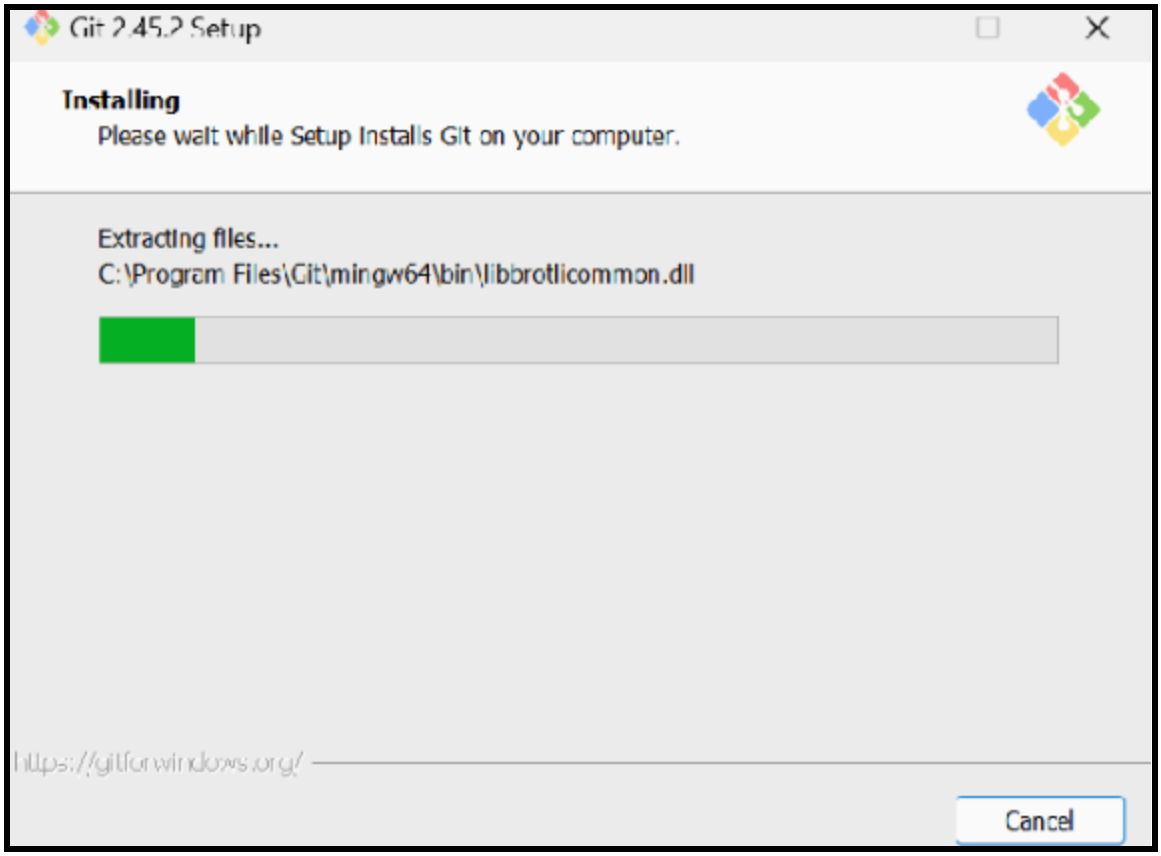
[ ] Only show new options | Back | Next | Cancel

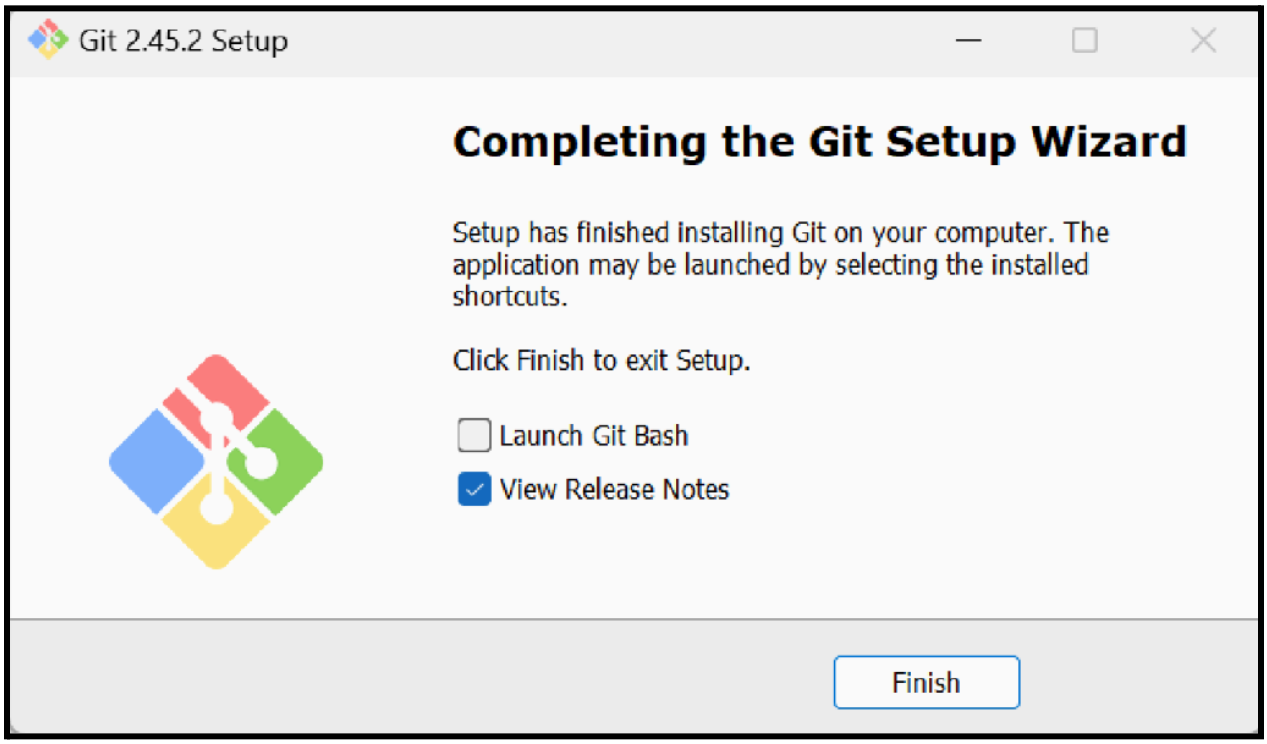

Git 2.52.0 Setup
Choose the default behavior of `git pull`
What should `git pull` do by default?
Fast-forward or merge
Fast-forward the current branch to the fetched branch when possible,
Rebase
Rebase the current branch onto the fetched branch. If there are no local
Only ever fast-forward
Fast-forward to the fetched branch. Fail if that is not possible.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Choose a credential helper
Which credential helper should be configured?
Git Credential Manager
Use the cross-platform Git Credential Manager.
here
None
Do not use a credential helper.
https://gitforwindows.org/
Only show new options
Back
Next
Cancel


Git 2.52.0 Setup
Configuring extra options
Which features would you like to enable?
Enable file system caching
File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant
Enable symbolic links
Enable symbolic links (requires the SeCreateSymbolicLink permission).
https://gitforwindows.org/
Only show new options
Back
Install
Cancel


Git 2.45.2 Setup
Configuring experimental options
These features are developed actively. Would you like to try them?
Enable experimental support for pseudo consoles.
This allows running native console programs like Node or Python in a Git Bash window without using winpty, but is unfortunately not quite stable yet.
Enable experimental built-in file system monitor
(NEW!) Automatically run a built-in file system watcher, to speed up common operations such as `git status`, `git add`, `git commit`, etc in worktrees containing many files.
Back
Install
Cancel


Git 2.45.2 Setup
Installing
Please wait while Setup installs Git on your computer.
Extracting files...
C:\Program Files\Git\mingw64\bin\libbrotlicommon.dll
https://gitforwindows.org/
Cancel


Git 2.45.2 Setup
Completing the Git Setup Wizard
Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch Git Bash
View Release Notes
Finish

Git Commands

1. **git --version**

   Displays the currently installed version of Git. Useful for ensuring you have the correct version installed.

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git --version
git version 2.47.0.windows.2

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

2. **git init**

   Initializes a new Git repository in the current directory, setting up all the necessary files and structures.

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git init
Initialized empty Git repository in C:/Users/MOAD/Desktop/Nelson Nadar 21/.git/

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

3. **git status**

   Shows the current state of the working directory and staging area, indicating which changes have been staged, which haven't, and which files aren't being tracked by Git.

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

4. git config

Used to set configuration values for your Git installation, such as your username, email, and other settings. Can be used at the global or local repository level.

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git config -v
error: unknown switch `v'
usage: git config list [<file-option>] [<display-option>] [--includes]
   or: git config get [<file-option>] [<display-option>] [--includes] [--all] [--regexp] [--v
alue=<value>] [--fixed-value] [--default=<default>] <name>
   or: git config set [<file-option>] [--type=<type>] [--all] [--value=<value>] [--fixed-valu
e] <name> <value>
   or: git config unset [<file-option>] [--all] [--value=<value>] [--fixed-value] <name>
   or: git config rename-section [<file-option>] <old-name> <new-name>
   or: git config remove-section [<file-option>] <name>
   or: git config edit [<file-option>]
   or: git config [<file-option>] --get-colorbool <name> [<stdout-is-tty>]

Config file location
    --[no-]global         use global config file
    --[no-]system         use system config file
    --[no-]local          use repository config file
    --[no-]worktree       use per-worktree config file
    -f, --[no-]file <file>
                          use given config file
    --[no-]blob <blob-id> read config from given blob object

Action
    --get                 get value: name [<value-pattern>]
    --get-all             get all values: key [<value-pattern>]
    --get-regexp          get values for regexp: name-regex [<value-pattern>]
    --get-urlmatch        get value specific for the URL: section[.var] URL
    --replace-all         replace all matching variables: name value [<value-pattern>]
    --add                 add a new variable: name value
    --unset               remove a variable: name [<value-pattern>]
    --unset-all           remove all matches: name [<value-pattern>]
    --rename-section      rename section: old-name new-name
    --remove-section      remove a section: name
    -l, --list            list all
    -e, --edit            open an editor
    --get-color           find the color configured: slot [<default>]
    --get-colorbool       find the color setting: slot [<stdout-is-tty>]

Display options
    -z, --[no-]null       terminate values with NUL byte
    --[no-]name-only      show variable names only
    --[no-]show-origin    show origin of config (file, standard input, blob, command line)
    --[no-]show-scope     show scope of config (worktree, local, global, system, command)
    --[no-]show-names     show config keys in addition to their values
```

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git config --global user.name "Nelson Puthuraja Nadar"

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git config --global user.email "crunchycurry69@student.sfit.ac.in"

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git config --global --list
user.email=crunchycurry69@student.sfit.ac.in
user.name=Nelson Puthuraja Nadar

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

5. **git add**

   Adds changes in the working directory to the staging area, preparing them to be included in the next commit. You can add specific files or use patterns.

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git add 1.txt

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   1.txt


MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ touch 2.txt 3.txt

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   1.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        2.txt
        3.txt
```

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git add .

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   1.txt
        new file:   2.txt
        new file:   3.txt


MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

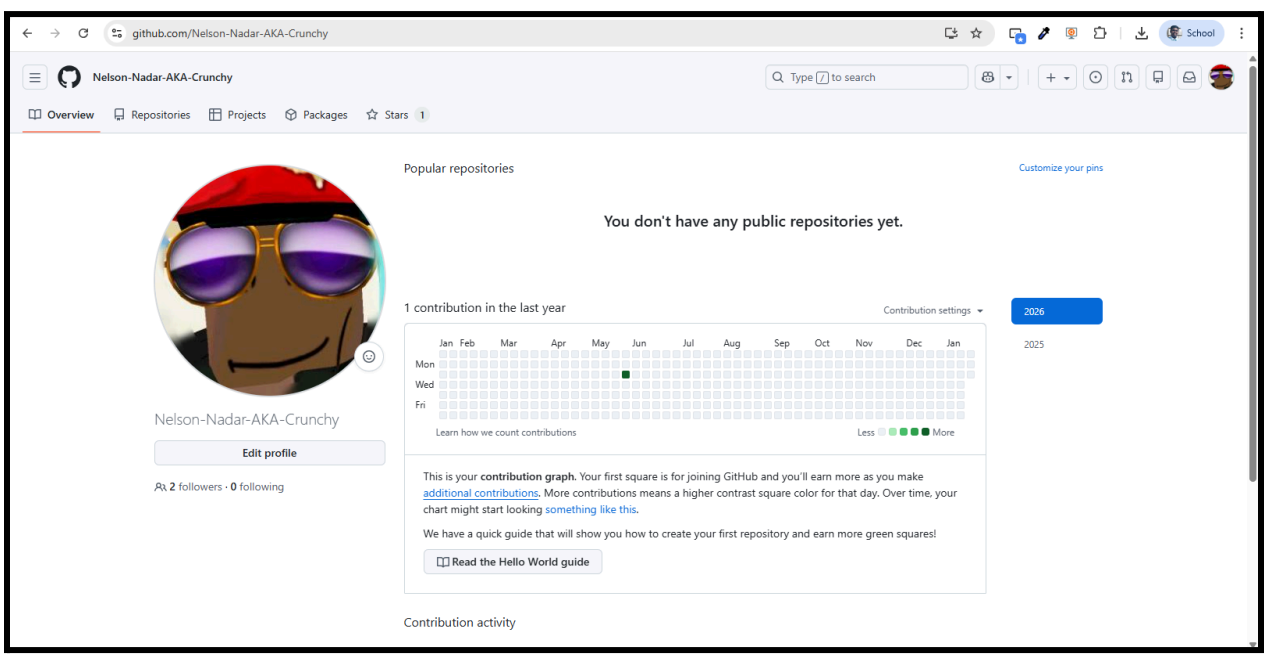
6. **git commit**

   Records the changes in the staging area into the repository's history, creating a new commit. You typically include a message to describe the changes made.

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git commit -m "First Commit"
[master (root-commit) 3d4d09b] First Commit
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 1.txt
 create mode 100644 2.txt
 create mode 100644 3.txt

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ touch 4.txt

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git add .

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git commit -m "Second Commit"
[master e5b5f56] Second Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 4.txt
```

7. **git log**

   Displays a log of commits made to the repository, showing the commit ID, author, date, and message. Useful for reviewing the project's history.

```
MINGW64:/c/Users/MOAD/Desktop/Nelson Nadar 21

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git log
commit e5b5f56776b9beb91ca4d7a9f7654003ef56117b (HEAD -> master)
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Jan 20 14:16:34 2026 +0530

    Second Commit

commit 3d4d09b2b82647160e2512dacbed6c5988ff46d7
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Jan 20 14:15:52 2026 +0530

    First Commit

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git log --oneline
e5b5f56 (HEAD -> master) Second Commit
3d4d09b First Commit

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git log 3d4d09b
commit 3d4d09b2b82647160e2512dacbed6c5988ff46d7
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Jan 20 14:15:52 2026 +0530

    First Commit

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

Github Account Creation